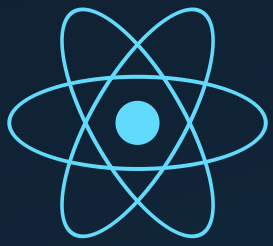

A COMPLETE COURSE BOOK

React.js

From Zero to Hero

A hands-on curriculum for teaching modern React

18 Chapters · 7 Parts · Exercises & Capstone Project

STUDENT EDITION

Table of Contents

ORIENTATION

How to Use This Course

This book is a complete, classroom-ready curriculum for teaching **React.js** to students who are starting from zero. It assumes only a working knowledge of HTML, CSS, and basic JavaScript. Everything else — components, hooks, routing, state management, testing, and deployment-ready patterns — is built up step by step.

Who This Course Is For

- **Teachers and instructors** who need a structured syllabus with explanations, code examples, and exercises.
- **Self-taught learners** who want a single guided path from "What is React?" to building and optimizing real applications.
- **Bootcamp and university students** in web-development tracks.

Course Structure

The course is organized into seven parts. Each part ends with skills that feed directly into the next, and the course finishes with a capstone project that combines everything.

Table 0-1 Course roadmap at a glance

Part	Focus	Outcome
I. Getting Started	What React is, tooling, first app	Student can scaffold and run a React project
II. Core Fundamentals	JSX, components, props, styling	Student can build static, reusable UI
III. Interactivity	Events, state, lists, forms	Student can build interactive UI
IV. Side Effects & Data	useEffect, data fetching	Student can connect UI to real APIs
V. Advanced Hooks	useRef, useMemo, useReducer, Context, custom hooks	Student can architect shared logic
VI. Real-World Skills	Routing, global state, performance, testing	Student can ship production-grade apps
VII. Capstone	Full project build	Portfolio-ready application

Conventions Used in This Book

Code appears in monospace blocks. Type it out yourself — never copy-paste when learning.

```
// This is a code example
function Hello() {
    return <h1>Hello, React!</h1>;
}
```

Four kinds of boxed content appear throughout the chapters:

> **Definition.** Precise explanations of key terms you must remember.

> **Common Pitfall.** Mistakes beginners make constantly — read these carefully.

> **Exercises.** Hands-on tasks at the end of each chapter. Assign them; mastery comes from doing.

KEY TAKEAWAYS. A compressed summary of the chapter — ideal for revision and quizzes.

Suggested Pacing

For a classroom setting, one chapter per session works well (roughly 18 sessions). Suggested distribution: lecture/demo 40%, guided coding 40%, exercises 20%. The capstone typically takes two to three sessions.

PART I

Getting Started

What React is, why it exists, and how to go from an empty folder to a running application.

CHAPTER 1

Introduction to React

1.1 What Is React?

React (also called React.js or ReactJS) is an open-source JavaScript library for building user interfaces. It was created at Facebook (now Meta), released publicly in 2013, and is today maintained by Meta together with a large open-source community. React powers the interfaces of Facebook, Instagram, Netflix, Airbnb, and countless other products.

> **Definition.** React is a *declarative, component-based JavaScript library* for building user interfaces. You describe *what* the UI should look like for a given state, and React figures out *how* to update the browser's DOM to match.

Three ideas define React:

1. **Declarative.** Instead of imperatively telling the browser "find this element, change its text, add a class," you declare: "when the state is X, render this." React handles the DOM operations.
2. **Component-based.** UIs are built from small, isolated, reusable pieces called *components* — a Button, a Navbar, a ProductCard — that compose into complex screens.
3. **Learn once, write anywhere.** The same component model powers web apps (React DOM), mobile apps (React Native), and even VR and desktop apps.

1.2 The Problem React Solves

Consider a classic vanilla-JavaScript approach to updating a to-do list:

```
// Vanilla JS: manual DOM manipulation
const list = document.getElementById("todo-list");
const item = document.createElement("li");
item.textContent = "Buy milk";
item.classList.add("todo-item");
list.appendChild(item);
document.getElementById("count").textContent = list.children.length;
```

This works for tiny pages. But as an application grows, you must manually keep the DOM synchronized with your data everywhere: counters, filters, permissions, loading spinners. Every new feature multiplies the number of places the UI can drift out of sync with the underlying data. Bugs become "the button shows 3 items but the list shows 2."

React flips the model: **your data (state) is the single source of truth, and the UI is a pure function of that data.**

```
UI = f(state)
```

When state changes, React re-runs the function and updates only what actually changed in the DOM. You never touch `document.createElement` again.

1.3 The Virtual DOM

The browser's DOM is slow to update directly. React keeps a lightweight in-memory representation of the UI — the **Virtual DOM**. The update process, called **reconciliation**, works like this:

1. State changes in a component.
2. React re-renders the component tree, producing a *new* Virtual DOM tree.
3. React *diffs* the new tree against the previous one.
4. Only the minimal set of real DOM changes is applied (this step is called *committing*).

> **Clarification.** The Virtual DOM is not "magic speed." Its real value is the *programming model*: it lets you write simple declarative code while React batches and minimizes expensive DOM operations for you.

1.4 Single-Page Applications (SPAs)

React is most often used to build **Single-Page Applications**: the browser loads one HTML page, and JavaScript swaps the visible content as the user "navigates" — no full page reloads. This produces fast, app-like experiences (think Gmail or Notion). In Chapter 14 you will add client-side routing to build exactly this.

1.5 The React Ecosystem

React itself is deliberately small — it only renders UI. The ecosystem provides the rest:

Table 1-1 Key parts of the React ecosystem used in this course

Tool	Purpose	Chapter
Vite	Development server & build tool	2
React Router	Client-side navigation between pages	14
Redux Toolkit / Zustand	Global state management	15

Vitest + Testing Library	Unit and component testing	17
Next.js	Full-stack React framework (post-course step)	App. C

1.6 A Brief History (So You Sound Like You Know Things)

- **2011–2013** — Created by Jordan Walke at Facebook; open-sourced in May 2013.
- **2015** — React Native brings React to mobile.
- **2019** — **Hooks** (React 16.8) revolutionize how logic is written; function components become the standard. This course teaches hooks exclusively.
- **2022** — React 18 adds concurrent rendering and automatic batching.
- **2024** — React 19 stabilizes Actions, the React Compiler direction, and improved server components support.

Common Pitfall. Old tutorials teach *class components* (`class App extends React.Component`). They still work, but modern React is written with *function components and hooks*. All code in this course uses the modern style. If you inherit legacy code, the concepts map over one-to-one.

Exercises 1.

1. In your own words, explain the difference between imperative and declarative UI code to a classmate.
2. Pick a website you use daily. Identify five visual "components" you could extract (e.g., a search bar, a card, an avatar).
3. Explain in two sentences what problem the Virtual DOM solves.

KEY TAKEAWAYS

- React is a declarative, component-based UI library created at Meta.
- UI is a function of state: `UI = f(state)`. Change state, and React updates the DOM.
- The Virtual DOM + reconciliation lets React apply minimal DOM updates efficiently.
- Modern React = function components + hooks.

CHAPTER 2

Environment Setup & Your First App

2.1 What You Need Installed

React development requires **Node.js**, which includes the `npm` package manager. Download the LTS version from `nodejs.org`, then verify in a terminal:

```
node --version    # e.g. v20.x.x — any modern LTS works
npm --version     # e.g. 10.x.x
```

You also need a code editor — **Visual Studio Code** is the de-facto standard. Recommended extensions: *ES7+ React/Redux snippets* and *Prettier*.

2.2 Scaffolding with Vite

Vite is the modern standard tool for starting React projects: it starts a dev server instantly and bundles for production. Create your first app:

```
npm create vite@latest my-first-app -- --template react
cd my-first-app
npm install
npm run dev
```

Open the printed URL (usually `http://localhost:5173`). You should see the Vite + React welcome page. Congratulations — you are running React.

> **Definition.Hot Module Replacement (HMR)**: when you save a file, Vite updates the running app in the browser *without a full reload* — often preserving component state. This makes the feedback loop nearly instant.

2.3 Project Structure, Explained

```
my-first-app/
├── node_modules/      # Installed dependencies (never edit)
├── public/            # Static assets served as-is
├── src/
│   ├── assets/        # Images, fonts imported by code
│   ├── App.jsx        # Your root component — start here
│   ├── App.css        # Styles for App
│   ├── main.jsx       # Entry point: mounts React into the page
│   └── index.css      # Global styles
├── index.html         # The single HTML page (the SPA shell)
├── package.json       # Project metadata & dependency list
└── vite.config.js     # Vite configuration
```

Two files matter most right now. First, the HTML shell — note it is almost empty:

```
<!-- index.html -->
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

Second, the entry point that mounts React into that empty `div`:

```
// src/main.jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Everything React renders lives inside `<div id="root">`. The `App` component is the root of your component tree.

> **Note.** `<StrictMode>` is a development-only helper. It intentionally double-invokes certain functions to surface bugs (like impure renders) early. It does nothing in production. You will see its effects in Chapter 9.

2.4 Writing Your First Component

Replace the contents of `src/App.jsx` with:

```
// src/App.jsx
function App() {
  const course = "React.js — From Zero to Hero";
  return (
    <main>
      <h1>{course}</h1>
      <p>My first React component is alive.</p>
    </main>
  );
}

export default App;
```

Save the file and watch the browser update instantly. You just wrote:

- a **function component** — a JavaScript function returning markup,
- **JSX** — the HTML-like syntax inside JavaScript (the subject of Chapter 3),
- an **expression slot** `{course}` that injects a JavaScript value into the markup.

2.5 Essential npm Commands

Table 2-1 npm commands you will use daily

Command	What it does
`npm run dev`	Start the dev server with HMR
`npm run build`	Create an optimized production bundle in `dist/`
`npm run preview`	Serve the production build locally to verify it
`npm install <pkg>`	Add a dependency (e.g. `npm install react-router-dom`)

> **Common Pitfall.** If `npm create vite` fails with a permissions error, do not use `sudo`. Fix npm's global directory permissions or use `npx` instead. And never commit `node_modules` to Git — commit `package.json` and `package-lock.json`; anyone can recreate the folder with `npm install`.

Exercises 2.

1. Scaffold a project called `react-playground` and run it.
2. Modify `App.jsx` to display your name and today's date (hint: `{new Date().toDateString()}`).
3. Delete the contents of the `return` and type your own markup from memory — a heading, a paragraph, and an unordered list of three hobbies.
4. Run `npm run build`, then `npm run preview`, and confirm the production version works.

KEY TAKEAWAYS

- Node.js + npm are the foundation; Vite is the standard React scaffolding tool.
- `main.jsx` mounts the root `App` component into `#root` in `index.html`.
- A component is a function that returns JSX.
- HMR gives near-instant feedback while you develop.

PART II

Core Fundamentals

JSX, components, props, and styling — the vocabulary every React developer uses all day, every day.

CHAPTER 3

JSX — JavaScript XML

3.1 What Is JSX?

JSX is a syntax extension for JavaScript that lets you write HTML-like markup inside your JavaScript files. It is not valid JavaScript — Vite compiles it into regular function calls before the browser ever sees it.

```
// You write this JSX:
const element = <h1 className="title">Hello</h1>;

// It compiles (conceptually) to:
const element = React.createElement("h1", { className: "title" }, "Hello");
```

JSX is optional — you could write `createElement` calls by hand — but nobody does, because JSX reads like the UI it produces.

3.2 Embedding Expressions with Curly Braces

Any valid JavaScript *expression* can be embedded in JSX with `{ }`:

```
function Profile() {
  const name = "Ada Lovelace";
  const year = 1843;
  return (
    <div>
      <h2>{name}</h2>
      <p>Published the first algorithm in {year}.</p>
      <p>That was {new Date().getFullYear() - year} years ago.</p>
      <p>Uppercase: {name.toUpperCase()}</p>
    </div>
  );
}
```

> **Common Pitfall.** Curly braces accept *expressions*, not *statements*. You cannot put `if`, `for`, or `while` directly inside JSX. Use the ternary operator (`cond ? a : b`), `&&`, or compute values *before* the `return`. See Chapter 7.

3.3 The Rules of JSX

1. Return a single root element

A component must return one parent element. Wrap siblings in a tag — or in a **Fragment** (`<>...</>`) that adds no extra DOM node:

```
// Wrong — two roots:
return (
  <h1>Title</h1>
  <p>Text</p>
);

// Correct — Fragment wrapper:
return (
  <>
    <h1>Title</h1>
    <p>Text</p>
  </>
);
```

2. Close every tag

JSX is XML-flavored: `<br>` must be `<br />` , `<img>` must be `<img />` , and every opening tag needs its closing tag.

3. camelCase attributes

JSX attributes are JavaScript properties, so most are camelCase:

Table 3-1 HTML attributes vs JSX props

HTML	JSX	Why
`class`	`className`	`class` is a reserved JS keyword
`for`	`htmlFor`	`for` is a reserved JS keyword
`onclick`	`onClick`	JS event-handler convention
`tabindex`	`tabIndex`	DOM property name
`style="color:red"`	`style={{ color: "red" }}`	Style takes an object, not a string

3.4 JSX Under the Hood

A JSX element is just a plain JavaScript object describing what should appear on screen:

```
// <h1 className="title">Hello</h1> becomes roughly:
{
  type: "h1",
  props: { className: "title", children: "Hello" }
}
```

React reads these objects (the Virtual DOM tree), compares them between renders, and updates the real DOM to match. This is why "rendering" in React simply means *calling your function and reading the objects it returns*.

3.5 Comments in JSX

```
function App() {
  // A normal JS comment, outside JSX
  return (
    <div>
      {/* A JSX comment must live inside braces */}
      <h1>Hello</h1>
    </div>
  );
}
```

Exercises 3.

1. Fix this code (three mistakes): `return <h1 class="big">Hi {name}</h1> <br> <p>Bye</p>`
2. Create a component that stores your name and age in variables and renders a sentence computing your age in months.
3. Render the result of `["a","b","c"].join(" + ")` inside a paragraph. What do you predict you will see? Verify.
4. Explain to a classmate why `style` needs double curly braces: `style={{ color: "red" }}`.

KEY TAKEAWAYS

- JSX = HTML-like syntax that compiles to JavaScript function calls.
- `{ }` embeds any expression; statements are not allowed.
- One root element (use Fragments), close all tags, camelCase attributes.
- JSX elements are plain objects — the Virtual DOM.

CHAPTER 4

Components & Props

4.1 Components: The Building Blocks

A **component** is a reusable, independent piece of UI. In modern React, a component is simply a JavaScript function whose name starts with a capital letter and that returns JSX.

```
// Greeting.jsx
function Greeting() {
  return <h2>Welcome to the course!</h2>;
}
export default Greeting;
```

```
// App.jsx — composing components like HTML tags
import Greeting from "./Greeting";

function App() {
  return (
    <main>
      <Greeting />
      <Greeting />
      <Greeting />
    </main>
  );
}
```

> **Definition.Composition** means building complex UI by nesting small components inside larger ones — the core design philosophy of React.

> **Common Pitfall.** Component names must be **PascalCase**. `<greeting />` is treated as a (nonexistent) HTML tag and renders nothing useful. Also: define components at the top level of a file — never define a component *inside* another component, which remounts it every render and loses state.

4.2 Props: Passing Data In

Props (properties) are how a parent component passes data down to a child. They are the component's arguments — a single object the function receives.

```
// UserCard.jsx
function UserCard(props) {
  return (
    <article className="card">
      <h3>{props.name}</h3>
      <p>Role: {props.role}</p>
    </article>
  );
}

// App.jsx — props look like HTML attributes
function App() {
  return (
    <>
      <UserCard name="Grace Hopper" role="Compiler Pioneer" />
      <UserCard name="Alan Turing" role="Computer Scientist" />
    </>
  );
}
```

The idiomatic style *destructures* props in the signature, often with default values:

```
function UserCard({ name, role = "Student" }) {
  return (
    <article className="card">
      <h3>{name}</h3>
      <p>Role: {role}</p>
    </article>
  );
}
```

4.3 The Special children Prop

Whatever you nest *between* a component's opening and closing tags arrives as `props.children`. This enables wrapper components:

```
function Panel({ title, children }) {
  return (
    <section className="panel">
      <h3>{title}</h3>
      <div className="panel-body">{children}</div>
    </section>
  );
}

// Usage — anything can go inside:
<Panel title="Announcements">
  <p>Class is cancelled on Friday.</p>
  <button>Acknowledge</button>
</Panel>
```

4.4 Props Flow One Way

React enforces **one-way data flow**: data travels down the tree from parent to child via props. A child never modifies its own props.

> **Definition.** Props are **read-only** (immutable) inside the receiving component. If a child needs to change something, the change must happen in the parent — typically via state and a callback function passed down as a prop (covered in Chapter 6).

4.5 Splitting a UI Into Components

A practical rule of thumb: if a piece of UI is used more than once, or a component grows past roughly a screen of code, extract it. A social-media post might decompose like this:

```
App
└── Feed
    └── Post (repeated per item)
        ├── PostHeader (avatar, author, time)
        ├── PostBody   (text, image)
        └── PostActions (like, comment, share)
```

Exercises 4.

1. Build a `ProductCard` component receiving `name`, `price`, and `inStock` props; render it three times with different data.
2. Add a default prop: if `inStock` is omitted, treat the product as in stock.
3. Build a `Layout` component using `children` that wraps any content in a header ("My Shop") and footer ("© 2026").
4. Refactor your solution so `ProductCard` uses a smaller `PriceTag` component internally.

KEY TAKEAWAYS

- Components are capitalized functions returning JSX; compose them like Lego bricks.
- Props are the component's input object — destructure them and set defaults.
- `children` enables flexible wrapper components.
- Data flows down; props are read-only.

CHAPTER 5

Styling in React

React does not prescribe one styling solution. This chapter surveys the four mainstream approaches and tells you when to use each.

5.1 Plain CSS Stylesheets

The default Vite approach: write normal CSS, import it into a component file.

```
/* Button.css */
.btn { padding: 8px 16px; border: none; background: #0d6efd; color: #fff; }
.btn:hover { background: #0b5ed7; }
```

```
import "./Button.css";
function Button({ label }) {
  return <button className="btn">{label}</button>;
}
```

Pros: zero learning curve, full CSS power. **Cons:** all classes are global — name collisions across files are common. Adopt a naming convention (BEM, or prefixing by component).

5.2 Inline Styles

The `style` prop takes a JavaScript object with camelCased property names:

```
function Badge({ count }) {
  const style = {
    backgroundColor: count > 0 ? "#dc3545" : "#6c757d",
    color: "white",
    borderRadius: "12px",
    padding: "2px 10px",
  };
  return <span style={style}>{count}</span>;
}
```

Pros: dynamic values are trivial. **Cons:** no pseudo-classes (`:hover`), no media queries, no reuse. Best for small dynamic touches, not full styling.

5.3 CSS Modules

Name a file `Card.module.css` and Vite scopes its classes to that file automatically — no collisions, still plain CSS.

```
/* Card.module.css */
.card { border: 1px solid #ddd; padding: 16px; }
.title { font-weight: bold; }
```

```
import styles from "./Card.module.css";

function Card({ title }) {
  return (
    <div className={styles.card}>
      <p className={styles.title}>{title}</p>
    </div>
  );
}
```

At build time `styles.card` becomes a unique class like `Card_card_a8f31`. **This is the recommended default** for course projects: real CSS, zero global-scope headaches.

5.4 Utility-First CSS (Tailwind)

Tailwind CSS provides atomic utility classes you compose directly in markup:

```
<button className="rounded bg-blue-600 px-4 py-2 text-white hover:bg-blue-700">
  Save
</button>
```

Pros: extremely fast iteration, no naming things. **Cons:** verbose markup, a build setup to learn. Widely used in industry — worth teaching after students are comfortable with plain CSS.

5.5 Conditional Classes

A recurring need: apply a class only when a condition holds. Use template literals or the tiny `clsx` library:

```
<li className={done ? "todo done" : "todo"}>{text}</li>

// with clsx (npm install clsx)
<li className={clsx("todo", done && "done", selected && "selected")}>
```

Table 5-1 Choosing a styling approach

Approach	Scoping	Dynamic styles	Best for
Plain CSS	Global	Via JS class toggling	Small projects, teaching CSS itself
Inline styles	Per element	Excellent	Quick dynamic values only
CSS Modules	Automatic	Via class toggling	Most course & production apps
Tailwind	N/A (utilities)	Via conditional classes	Rapid UI development, teams

Exercises 5.

1. Style your `ProductCard` from Chapter 4 with a CSS Module: border, padding, and a red "Out of stock" label when `inStock` is false.
2. Add a `:hover` effect — notice it is impossible with inline styles.
3. Create a `Badge` whose background color is passed as a prop and applied via inline style.

KEY TAKEAWAYS

- React is styling-agnostic: plain CSS, inline objects, CSS Modules, or Tailwind all work.
- CSS Modules give scoped, real CSS with zero config in Vite — the course default.
- Inline styles are JS objects for dynamic values; they cannot do hover/media queries.

PART III

Interactivity

State, events, lists, and forms — where static pages become living applications.

CHAPTER 6

Events & State with useState

6.1 Handling Events

React events are camelCase props receiving a *function* — not a string, and not a function call:

```
function ClickDemo() {
  function handleClick() {
    alert("Button clicked!");
  }
  return <button onClick={handleClick}>Click me</button>;
}
```

Common Pitfall. `onClick={handleClick()}` (with parentheses) *calls the function during render* and passes its return value — the alert fires immediately, and the click does nothing. Pass the function itself: `onClick={handleClick}`. To pass arguments, wrap it: `onClick={() => handleClick(id)}`.

Common events: `onClick`, `onChange`, `onSubmit`, `onKeyDown`, `onMouseEnter`. Handlers receive a *synthetic event* object, a cross-browser wrapper around the native event:

```
<input onChange={(event) => console.log(event.target.value)} />
```

6.2 Why Plain Variables Don't Work

```
function BrokenCounter() {
  let count = 0;               // a normal variable
  return (
    <button onClick={() => { count = count + 1; console.log(count); }}>
      Count: {count}
    </button>
  );
}
```

Click it: the console logs 1, 2, 3… but the screen never changes. Why? Because **React only re-renders a component when its state or props change**. A local variable mutation is invisible to React — and the variable resets on every render anyway. This is the single most important lesson of the chapter.

6.3 useState — Memory for Components

The `useState` hook gives a component a piece of remembered data plus a function to update it:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);   // [value, setter] = useState(initial)

  return (
    <div>
      <p>You clicked {count} times.</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

> **Definition.** A **hook** is a special function (always prefixed with `use`) that lets a function component "hook into" React features. `useState` returns a pair: the current state value and a setter. Calling the setter tells React: "this data changed — re-render me."

The render cycle: *event → setter → re-render → new JSX → React updates the DOM.*

6.4 Updating State Based on Previous State

```
// Risky: both read the same `count` snapshot → only +1 total
setCount(count + 1);
setCount(count + 1);

// Correct: functional updates queue on the latest value → +2 total
setCount((prev) => prev + 1);
setCount((prev) => prev + 1);
```

Rule: whenever the new state depends on the old state, use the functional form `setX((prev) => ...)`.

6.5 State with Objects and Arrays — Immutability

State must be updated **immutably**: never mutate the existing object/array; always create a new one. React compares references to detect change — mutating in place keeps the same reference, and React may skip the re-render.

```
const [user, setUser] = useState({ name: "Ada", age: 36 });

// Wrong — mutates the existing object:
user.age = 37;

// Correct — spread into a new object:
setUser({ ...user, age: 37 });
```

```
const [todos, setTodos] = useState(["learn JSX"]);

// Add      → new array with spread
setTodos([...todos, "learn state"]);
// Remove   → filter creates a new array
setTodos(todos.filter((t) => t !== "learn JSX"));
// Update   → map creates a new array
setTodos(todos.map((t) => (t === "learn JSX" ? "master JSX" : t)));
```

Table 6-1 Immutable update patterns to memorize

Operation	Array	Object
Add / set	`[...arr, item]`	`{ ...obj, key: value }`
Remove	`arr.filter(...)`	destructure-rest, or omit key in a new object
Update	`arr.map(...)`	`{ ...obj, key: newValue }`

6.6 The Rules of Hooks

1. **Only call hooks at the top level** of your component — never inside `if`, loops, or nested functions. React relies on the consistent call order to match state to hooks between renders.
2. **Only call hooks from React functions** — components or custom hooks (Chapter 13), never from regular functions.

6.7 Lifting State Up

When two sibling components need the same data, move the state to their *closest common parent* and pass it down via props — with a callback so children can request changes:

```
function App() {
  const [theme, setTheme] = useState("light");
  return (
    <>
      <Toolbar theme={theme} onToggle={() => setTheme(theme === "light" ? "dark" :
"light")} />
      <Content theme={theme} />
    </>
  );
}
```

This completes the one-way data-flow picture from Chapter 4: **state lives up, props flow down, events flow up** through callbacks.

Exercises 6.

1. Build a counter with +1, −1, and reset buttons. Disable −1 when the count is 0.
2. Build a "like" button that toggles between ♡ and ♥ and shows a like count.
3. Build a shopping list: an input + button adds items (immutable add), and clicking an item removes it (`filter`).
4. Create two `TemperatureDisplay` components (Celsius and Fahrenheit) sharing one state lifted into their parent.

KEY TAKEAWAYS

- Events take function references: `onClick={fn}` , never `fn()` .
- Only state/props changes trigger re-renders — plain variables are invisible to React.
- `const [value, setValue] = useState(initial)` ; use functional updates when depending on previous state.
- Update objects/arrays immutably: spread, filter, map — never mutate.
- Hooks: top level only, React functions only.

CHAPTER 7

Conditional Rendering & Lists

7.1 Conditional Rendering

Since JSX accepts expressions, you choose *what* to render with JavaScript's conditional expressions.

Ternary operator — if/else

```
function Greeting({ isLoggedIn }) {
  return <h2>{isLoggedIn ? "Welcome back!" : "Please sign in."}</h2>;
}
```

Logical && — if only

```
function Inbox({ unread }) {
  return (
    <div>
      <h2>Inbox</h2>
      {unread > 0 && <p>You have {unread} unread messages.</p>}
    </div>
  );
}
```

Common Pitfall. `{count && <Badge />}` renders the literal number **0** when `count` is 0, because `0` is falsy but still renderable. Always force a real boolean: `{count > 0 && <Badge />}` or `{!!count && <Badge />}`.

Early returns and variables

```
function Dashboard({ user }) {
  if (!user) return <p>Loading…</p>;       // early return pattern

  let status;
  if (user.isAdmin) status = <AdminPanel />;
  else status = <UserPanel />;

  return <main>{status}</main>;
}
```

7.2 Rendering Lists with map()

Arrays transform into JSX arrays with `map()`:

```
function ShoppingList({ items }) {
  return (
    <ul>
      {items.map((item) => (
        <li key={item.id}>{item.name} — ${item.price}</li>
      ))}
    </ul>
  );
}
```

And lists of components are just as natural:

```
{products.map((p) => <ProductCard key={p.id} name={p.name} price={p.price} />)}
```

7.3 Keys: Giving Elements an Identity

> **Definition.** A **key** is a special string prop React uses to identify which list item is which between renders. With stable keys, React can reorder, insert, or delete DOM nodes minimally and preserve per-item state correctly.

Rules for keys:

- Keys must be **unique among siblings** (not globally).
- Keys must be **stable** — a database id is ideal.
- **Do not use the array index as a key** when the list can reorder, insert, or delete — indices shift, and React will match state to the wrong rows, producing subtle, maddening bugs. Index keys are acceptable only for truly static lists.

7.4 Putting It Together: A Filterable List

```
import { useState } from "react";

const ALL_PRODUCTS = [
  { id: 1, name: "Keyboard", category: "tech" },
  { id: 2, name: "Notebook", category: "office" },
  { id: 3, name: "Mouse", category: "tech" },
];

function ProductFilter() {
  const [category, setCategory] = useState("all");

  const visible = ALL_PRODUCTS.filter(
    (p) => category === "all" || p.category === category
  );

  return (
    <div>
      <button onClick={() => setCategory("all")}>All</button>
      <button onClick={() => setCategory("tech")}>Tech</button>
      <button onClick={() => setCategory("office")}>Office</button>

      {visible.length === 0 ? (
        <p>No products in this category.</p>
      ) : (
        <ul>
          {visible.map((p) => <li key={p.id}>{p.name}</li>)}
        </ul>
      )}
    </div>
  );
}
```

Notice the pattern: **derive** (`visible`) from state rather than storing filtered copies. Deriving during render keeps one source of truth.

Exercises 7.

1. Render a list of students from an array of objects; show "Honor roll" next to anyone with a grade ≥ 90 using `&&`.
2. Add a toggle button that switches between "list view" and "grid view" of the same data (ternary on a state string).
3. Build a search box that filters a list of countries as you type (combines Chapter 6 state with this chapter's `filter` + `map`).
4. Demonstrate the index-key bug: build a list of inputs, key them by index, prepend an item, and watch the input values attach to the wrong rows. Fix it with stable ids.

KEY TAKEAWAYS

- Use ternaries for if/else, `&&` for if-only, early returns for guards.
- Beware `{0 && ...}` rendering a literal 0 — compare explicitly.
- `map()` turns arrays into JSX; every item needs a stable, unique `key`.
- Derive filtered/sorted views during render instead of duplicating state.

CHAPTER 8

Forms & Controlled Components

8.1 Controlled Inputs

In React, form inputs are usually **controlled**: their value comes from state, and every keystroke updates that state via `onChange`. The input displays what state says — a single source of truth.

```
import { useState } from "react";

function NameForm() {
  const [name, setName] = useState("");

  return (
    <div>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Your name"
      />
      <p>Hello, {name || "stranger"}!</p>
    </div>
  );
}
```

> **Definition.** A **controlled component** is an input whose value is driven by React state (`value` + `onChange`). An **uncontrolled component** lets the DOM hold the value, read later via a ref (Chapter 1 1). Default to controlled.

8.2 One Handler for Many Inputs

Use the input's `name` attribute and computed keys to scale to real forms:

```
const [form, setForm] = useState({ email: "", password: "", remember: false });

function handleChange(e) {
  const { name, value, type, checked } = e.target;
  setForm((prev) => ({ ...prev, [name]: type === "checkbox" ? checked : value }));
}

<input name="email"    value={form.email}    onChange={handleChange} />
<input name="password" value={form.password} onChange={handleChange} type="password"
/>
<input name="remember" checked={form.remember} onChange={handleChange}
type="checkbox" />
```

8.3 Other Input Types

Table 8-1 Controlling different input types

Element	Value prop	Read from event
`<input type="text">`, `<textarea>`	`value`	`e.target.value`
`<input type="checkbox">`	`checked`	`e.target.checked`
`<input type="radio">`	`checked`	`e.target.checked`
`<select>`	`value` on the `<select>`	`e.target.value`
`<input type="number">`	`value`	`e.target.value` (still a string!)

8.4 Submitting and Validating

```
function SignupForm() {
  const [form, setForm] = useState({ email: "", password: "" });
  const [errors, setErrors] = useState({});

  function validate() {
    const next = {};
    if (!form.email.includes("@")) next.email = "Enter a valid email.";
    if (form.password.length < 8) next.password = "At least 8 characters.";
    return next;
  }

  function handleSubmit(e) {
    e.preventDefault();              // stop the browser's full-page reload
    const next = validate();
    setErrors(next);
    if (Object.keys(next).length === 0) {
      console.log("Submit to server:", form);
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="email" value={form.email}
        onChange={(e) => setForm({ ...form, email: e.target.value })} />
      {errors.email && <p className="error">{errors.email}</p>}

      <input name="password" type="password" value={form.password}
        onChange={(e) => setForm({ ...form, password: e.target.value })} />
      {errors.password && <p className="error">{errors.password}</p>}

      <button type="submit">Sign up</button>
    </form>
  );
}
```

Common Pitfall. Forgetting `e.preventDefault()` in `onSubmit` makes the browser reload the page, wiping your SPA's state. Also: inputs read as strings — convert with `Number(...)` before arithmetic.

8.5 Why Controlled Forms Win

- **Instant validation** — check on every keystroke, disable submit until valid.
- **Dynamic formatting** — force uppercase, mask phone numbers as the user types.

- **Derived UI** — live previews, character counters, matching-password checks.

For very large forms, libraries such as *React Hook Form* (uncontrolled-based, fewer re-renders) and *Zod* (schema validation) are industry standards — learn them after mastering the manual pattern above.

Exercises 8.

1. Build a login form (email + password) with validation errors under each field and a disabled submit button until valid.
2. Add a "show password" checkbox that toggles the input `type`.
3. Build a survey form using a `<select>`, radio buttons, and a checkbox — all controlled by one state object and one handler.
4. Add a character counter (e.g., "42 / 280") under a controlled `<textarea>`.

KEY TAKEAWAYS

- Controlled inputs: `value` from state, `onChange` updates state.
- One generic `handleChange` + `name` attributes scales to any form size.
- `e.preventDefault()` in submit handlers; validate before sending.
- Controlled forms enable live validation, formatting, and derived UI.

PART IV

Side Effects & Data

Reaching outside React: timers, subscriptions, and talking to real APIs.

CHAPTER 9

useEffect & the Component Lifecycle

9.1 What Is a Side Effect?

Rendering should be a *pure* calculation: props + state in, JSX out. But apps must also do things outside that calculation — fetch data, start timers, subscribe to sockets, change `document.title`, write to `localStorage`. These are **side effects**, and they belong in `useEffect`, never directly in the render body.

> **Definition.** `useEffect(setup, dependencies)` runs the *setup* function **after** React has rendered and committed to the DOM. The *dependency array* controls when it re-runs.

```
import { useEffect, useState } from "react";

function Clock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setTime(new Date()), 1000);
    return () => clearInterval(id);     // cleanup function
  }, []);                               // empty deps → run once on mount

  return <p>{time.toLocaleTimeString()}</p>;
}
```

9.2 The Dependency Array

Table 9-1 How dependencies control useEffect

Form	When the effect runs	Typical use
`useEffect(fn)` — no array	After *every* render	Rare; usually a bug
`useEffect(fn, [])`	Once, after the first render (mount)	Initial fetch, subscriptions, timers
`useEffect(fn, [a, b])`	On mount, and whenever `a` or `b` changes	Re-fetch when an id changes; sync with a prop

Every value used inside the effect that could change must appear in the array. The ESLint plugin `react-hooks` (included in Vite templates) warns when you miss one — obey it.

9.3 The Cleanup Function

Returning a function from `useEffect` registers a **cleanup** that React runs *before* re-running the effect and when the component unmounts. Cleanup prevents leaks and stale behavior:

```
useEffect(() => {
  const onResize = () => setWidth(window.innerWidth);
  window.addEventListener("resize", onResize);
  return () => window.removeEventListener("resize", onResize);  // no leaks
}, []);
```

9.4 The Lifecycle, in Hook Terms

Table 9-2 Component lifecycle phases mapped to hooks

Phase	Meaning	Hook pattern
Mount	Component appears on screen	`useEffect(fn, [])`
Update	Props/state change → re-render	`useEffect(fn, [dep])`
Unmount	Component is removed	Cleanup function returned from the effect

StrictMode Double-Effects. In development, `<StrictMode>` mounts → unmounts → remounts every component once, so your effect runs *twice*. This is intentional: it exposes missing cleanups (e.g., a duplicated interval or a double subscription). If your effect with proper cleanup is safe when run twice, it is correct. Production runs once.

9.5 When You Do NOT Need useEffect

The most overused hook. You do *not* need an effect to:

- **Derive data from state/props** — compute it during render: `const fullName = first + " " + last;`
- **Transform a list** — `filter` / `map` during render (memoize only if truly expensive, Chapter 11).
- **Respond to a user event** — put logic in the event handler, not an effect watching state.

Effects are for *synchronizing with systems outside React* (network, DOM APIs, timers, subscriptions). If nothing external is involved, an effect is probably the wrong tool.

Exercises 9.

1. Build the `Clock` above; then deliberately remove the cleanup, switch it off and on, and observe multiple intervals stacking up (check the seconds jumping by 2). Fix it.
2. Make `document.title` show a live notification count: `useEffect(..., [count])`.
3. Build a window-size tracker with proper `resize` listener cleanup.
4. Persist a text input to `localStorage` on every change, and initialize state from it.

KEY TAKEAWAYS

- `useEffect` synchronizes with external systems after render.
- The dependency array controls re-runs; the returned function cleans up.
- StrictMode's double-invocation in dev reveals missing cleanups.
- Derive-in-render beats effects; effects are only for outside-React systems.

CHAPTER 10

Fetching Data & Async Patterns

10.1 The Three States of Every Request

Every async operation in a UI has three faces — model all three explicitly:

- **Loading** — show a spinner/skeleton.
- **Error** — show a friendly message + retry.
- **Success** — show the data.

10.2 Fetch on Mount

```
import { useEffect, useState } from "react";

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    async function loadUsers() {
      try {
        const res = await fetch("https://jsonplaceholder.typicode.com/users", {
          signal: controller.signal,
        });
        if (!res.ok) throw new Error("HTTP " + res.status);
        const data = await res.json();
        setUsers(data);
      } catch (err) {
        if (err.name !== "AbortError") setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    loadUsers();
    return () => controller.abort();   // cancel if component unmounts
  }, []);

  if (loading) return <p>Loading users…</p>;
  if (error)   return <p>Failed to load: {error}</p>;
  return (
    <ul>
      {users.map((u) => <li key={u.id}>{u.name} — {u.email}</li>)}
    </ul>
  );
}
```

> **Definition.** `AbortController` cancels an in-flight `fetch`. Aborting in the effect's cleanup prevents setting state after unmount and discards stale responses when a request is superseded by a newer one.

10.3 The Race Condition Problem

Suppose the effect re-runs whenever a `userId` prop changes. The user clicks fast: request A (id 1) and request B (id 2) are both in flight. If A resolves *after* B, the screen shows stale data for user 1. The cleanup

fixes it:

```
useEffect(() => {
  let ignore = false;
  fetch("/api/users/" + userId)
    .then((r) => r.json())
    .then((data) => { if (!ignore) setUser(data); });
  return () => { ignore = true; };   // stale responses are ignored
}, [userId]);
```

10.4 POSTing Data

```
async function createTodo(title) {
  const res = await fetch("/api/todos", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, done: false }),
  });
  if (!res.ok) throw new Error("Save failed");
  return res.json();
}

// In an event handler (not an effect!):
async function handleAdd() {
  try {
    const saved = await createTodo(text);
    setTodos((prev) => [...prev, saved]);   // server returns the saved item
  } catch (err) {
    setError("Could not save. Try again.");
  }
}
```

> **Common Pitfall.** `fetch` only rejects on *network* failure. An HTTP 404 or 500 still "succeeds" — you must check `res.ok` yourself and throw. Forgetting this silently renders empty screens.

10.5 Data-Fetching Libraries

Manual fetch effects teach the fundamentals, but production apps usually adopt a data-fetching library that handles caching, deduping, retries, and background refresh:

- **TanStack Query (React Query)** — the industry standard for server state.
- **SWR** — lightweight, by Vercel.
- **Redux Toolkit Query** — if you already use Redux (Chapter 15).

```
// The same screen with TanStack Query:
const { data: users, isLoading, error } = useQuery({
  queryKey: ["users"],
  queryFn: () => fetch("/api/users").then((r) => r.json()),
});
```

Exercises 10.

1. Fetch and display posts from `jsonplaceholder.typicode.com/posts` with full loading/error handling.
2. Build a user search: typing a name re-fetches filtered results. Add the `ignore` flag and prove stale responses no longer overwrite fresh ones.
3. Add a "Retry" button to the error state.
4. Implement a comments section: GET comments for a post, POST a new comment, and append the server's response to local state.

KEY TAKEAWAYS

- Always model loading, error, and success states.
- Fetch in effects (reads) with `AbortController`; mutate in event handlers (writes).
- Check `res.ok` — fetch doesn't throw on HTTP errors.
- Guard against race conditions; consider TanStack Query for real apps.

PART V

Advanced Hooks & Patterns

Refs, memoization, reducers, context, and custom hooks — the toolkit of senior React developers.

CHAPTER 11

useRef, useMemo & useCallback

11.1 useRef — Values That Survive Renders Without Causing Them

`useRef` returns a mutable box, `{ current: value }`, with two superpowers:

1. Mutating `ref.current` does **not** trigger a re-render.
2. The box persists for the component's lifetime.

Use case 1: Accessing DOM elements

```
import { useRef } from "react";

function SearchBox() {
  const inputRef = useRef(null);

  return (
    <>
      <input ref={inputRef} placeholder="Search…" />
      <button onClick={() => inputRef.current.focus()}>Focus the input</button>
    </>
  );
}
```

Use case 2: Storing values across renders

```
function Stopwatch() {
  const [elapsed, setElapsed] = useState(0);
  const intervalRef = useRef(null);

  function start() {
    if (intervalRef.current) return;              // already running
    intervalRef.current = setInterval(() => setElapsed((s) => s + 1), 1000);
  }
  function stop() {
    clearInterval(intervalRef.current);
    intervalRef.current = null;
  }
  return (
    <>
      <p>{elapsed}s</p>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </>
  );
}
```

Table 11-1 useState vs useRef

	useState	**useRef**
Update triggers re-render?	Yes	No
Persists across renders?	Yes	Yes
Use for	Data shown in the UI	DOM nodes, timers, previous values, any mutable behind-the-scenes value

11.2 useMemo — Caching Expensive Calculations

`useMemo(fn, deps)` re-runs `fn` only when `deps` change and caches the result between renders:

```
import { useMemo, useState } from "react";

function ProductSearch({ products }) {
  const [query, setQuery] = useState("");

  const visible = useMemo(() => {
    console.log("filtering…");           // runs only when inputs change
    return products.filter((p) =>
      p.name.toLowerCase().includes(query.toLowerCase())
    );
  }, [products, query]);

  return ( /* render `visible` */ );
}
```

Common Pitfall. Do not wrap everything in `useMemo` "for performance." Memoization itself costs memory and comparison work. Reach for it only when a calculation is genuinely expensive (large lists, heavy transforms) or when a stable reference is needed for effect dependencies or memoized children. Measure first.

11.3 useCallback — Caching Functions

Functions are recreated on every render, so passing a fresh function to a memoized child (Chapter 16) defeats the memoization. `useCallback(fn, deps)` returns the *same function instance* until deps change:

```
const handleSelect = useCallback((id) => {
  setSelectedId(id);
}, []);   // stable forever — setSelectedId never changes

// Now <ExpensiveList onSelect={handleSelect} /> can skip re-rendering
```

Exercises 11.

1. Build the stopwatch; add a "lap" button that stores lap times in a ref-backed array and displays them.
2. Build a component that renders a 50,000-item list filtered by a query; wrap the filter in `useMemo` and log to prove it skips work on unrelated re-renders.
3. Display the *previous* value of a counter alongside the current one (hint: update a ref in an effect).

KEY TAKEAWAYS

- `useRef` : mutable, render-invisible storage — for DOM nodes and behind-the-scenes values.
- `useMemo` : caches expensive computations between renders.
- `useCallback` : caches function identity; pairs with memoized children.
- All three are optimizations — correctness first, measure, then memoize.

CHAPTER 12

useReducer & the Context API

12.1 When useState Gets Messy

As state grows — many related fields, updates that depend on each other — scattered `setState` calls become hard to follow. `useReducer` centralizes all update logic in one pure function.

> **Definition.** A **reducer** is a pure function `(state, action) => newState`. Components *dispatch* plain-object actions describing *what happened*; the reducer decides *how state changes*.

```
import { useReducer } from "react";

const initialState = { items: [], total: 0 };

function cartReducer(state, action) {
  switch (action.type) {
    case "added": {
      const items = [...state.items, action.product];
      return { items, total: state.total + action.product.price };
    }
    case "removed": {
      const product = state.items.find((p) => p.id === action.id);
      return {
        items: state.items.filter((p) => p.id !== action.id),
        total: state.total - product.price,
      };
    }
    case "cleared":
      return initialState;
    default:
      throw new Error("Unknown action: " + action.type);
  }
}

function Cart() {
  const [state, dispatch] = useReducer(cartReducer, initialState);

  return (
    <>
      <p>{state.items.length} items — ${state.total}</p>
      <button onClick={() => dispatch({ type: "added", product: { id: 1, price: 29 }
})}>
        Add $29 item
      </button>
      <button onClick={() => dispatch({ type: "cleared" })}>Clear</button>
    </>
  );
}
```

Benefits: update logic is testable in isolation (`expect(cartReducer(s, a)).toEqual(...)`), every change is an explicit named action, and complex transitions stay in one place. `useReducer` is also the conceptual foundation of Redux (Chapter 15).

12.2 Prop Drilling: The Problem Context Solves

Imagine a deeply nested tree where `App` holds the logged-in user, and a tiny `Avatar` five levels down needs it. With props alone, every intermediate component must forward `user` without using it — **prop drilling**. It clutters code and makes refactors painful.

12.3 The Context API

Context lets a component *broadcast* a value to everything below it, skipping the middle levels.

```
// 1. Create the context (usually in its own file)
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext(null);

// 2. Provide the value high in the tree
function App() {
  const [theme, setTheme] = useState("light");
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Page />      {/* any depth of components in between */}
    </ThemeContext.Provider>
  );
}

// 3. Consume it anywhere below — no props needed
function ThemeButton() {
  const { theme, setTheme } = useContext(ThemeContext);
  return (
    <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
      Current theme: {theme}
    </button>
  );
}
```

Common Pitfall. Context is **not** a state-management replacement. Every context change re-renders *all consuming components*, so high-frequency updates (mouse positions, form keystrokes) do not belong there. Use context for low-frequency, widely-needed values: theme, locale, authenticated user, and for dependency injection.

12.4 The Power Combo: Context + useReducer

Pairing them yields lightweight global state: the reducer owns the logic, context distributes state and dispatch:

```
const CartContext = createContext(null);

export function CartProvider({ children }) {
  const [state, dispatch] = useReducer(cartReducer, initialState);
  return (
    <CartContext.Provider value={{ state, dispatch }}>
      {children}
    </CartContext.Provider>
  );
}

export function useCart() {
  return useContext(CartContext);   // nice consumer API
}

// Anywhere in the tree:
// const { state, dispatch } = useCart();
```

Exercises 12.

1. Rewrite a to-do app from `useState` to `useReducer` with actions `added`, `toggled`, `deleted`.
2. Write unit tests (plain JS) for your reducer — this is the payoff.
3. Build a theme + language context provider; consume it in a header and a footer at different depths.
4. Combine both: a global to-do store via Context + useReducer, consumed by a form and a list in separate branches.

KEY TAKEAWAYS

- `useReducer` centralizes state transitions in a pure, testable function.
- Context broadcasts values down the tree, eliminating prop drilling.
- Context + useReducer = lightweight global state without libraries.
- Context is for low-frequency, widely-shared values — not everything.

CHAPTER 13

Custom Hooks

13.1 Logic Reuse in React

Components reuse *UI*. Custom hooks reuse *stateful logic*. A custom hook is simply a function whose name starts with `use` and that calls other hooks.

> **Definition.** A **custom hook** extracts stateful logic (state, effects, refs) into a reusable function. Every component that calls it gets its *own independent copy* of the state — the logic is shared, the state is not.

13.2 Building useLocalStorage

```
import { useEffect, useState } from "react";

function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const stored = localStorage.getItem(key);
    return stored !== null ? JSON.parse(stored) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];   // same interface as useState
}

// Usage — identical API to useState, but persistent:
function Settings() {
  const [username, setUsername] = useLocalStorage("username", "");
  return <input value={username} onChange={(e) => setUsername(e.target.value)} />;
}
```

13.3 Building useFetch

```
import { useEffect, useState } from "react";

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);

    fetch(url, { signal: controller.signal })
      .then((res) => {
        if (!res.ok) throw new Error("HTTP " + res.status);
        return res.json();
      })
      .then(setData)
      .catch((err) => { if (err.name !== "AbortError") setError(err.message); })
      .finally(() => setLoading(false));

    return () => controller.abort();
  }, [url]);

  return { data, loading, error };
}

// One line in any component:
const { data: posts, loading } = useFetch("/api/posts");
```

13.4 More Ideas and Design Tips

- `useWindowSize()` — tracks `{ width, height }` with a resize listener.
- `useDebounce(value, delay)` — returns the value only after the user stops typing; perfect for search inputs.
- `useToggle(initial)` — boolean state with a `toggle()` function.
- `usePrevious(value)` — returns the value from the previous render.

Design guidelines: name hooks `useX`; return objects for three or more values, arrays for useState-like pairs; keep hooks focused on one job; and compose hooks from other hooks just as you compose components.

Exercises 13.

1. Implement `useDebounce` and wire it to a search input that logs "API call" only 500 ms after typing stops.
2. Implement `useToggle` and refactor a modal open/close to use it.
3. Implement `useWindowSize` and render "Mobile" / "Desktop" based on a 768 px breakpoint.
4. Extract the data-fetching logic from your Chapter 10 exercises into `useFetch`.

KEY TAKEAWAYS

- Custom hooks share stateful *logic*; components share UI.
- A custom hook is just a function starting with `use` that calls hooks.
- Each caller gets independent state.
- useLocalStorage, useFetch, useDebounce — build your own toolbox.

PART VI

Real-World Application Skills

Routing, global state, performance, and testing — the skills that separate demos from shipped products.

CHAPTER 14

Routing with React Router

14.1 Client-Side Routing

In an SPA, "pages" are really components swapped by JavaScript. **React Router** syncs the URL with the visible component — giving users shareable links, working back/forward buttons, and bookmarkable pages without server round-trips.

```
npm install react-router-dom
```

14.2 Basic Setup

```
// main.jsx
import { BrowserRouter } from "react-router-dom";

createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

```
// App.jsx
import { Routes, Route, Link } from "react-router-dom";

function App() {
  return (
    <>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/products">Products</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/products" element={<Products />} />
        <Route path="*" element={<h1>404 — Not Found</h1>} />
      </Routes>
    </>
  );
}
```

> **Common Pitfall.** Never use `<a href="/about">` inside an SPA — it triggers a full page reload and destroys app state. Always use `<Link>` (or `<NavLink>`, which adds an `active` style to the current page's link).

14.3 Dynamic Routes & URL Parameters

```
<Route path="/products/:productId" element={<ProductDetail />} />

// ProductDetail.jsx — read the parameter:
import { useParams } from "react-router-dom";

function ProductDetail() {
  const { productId } = useParams();          // "/products/42" → "42"
  const { data: product, loading } = useFetch("/api/products/" + productId);
  /* render product… */
}
```

14.4 Programmatic Navigation & Query Strings

```
import { useNavigate, useSearchParams } from "react-router-dom";

const navigate = useNavigate();
navigate("/dashboard");        // go somewhere after login
navigate(-1);                  // back button

// Query strings: /search?q=react&page=2
const [searchParams, setSearchParams] = useSearchParams();
const query = searchParams.get("q");
setSearchParams({ q: "hooks", page: 1 });
```

14.5 Nested Routes & Layouts

Shared layouts (sidebars, headers) wrap nested pages via `<Outlet />`:

```
<Route path="/dashboard" element={<DashboardLayout />}>
  <Route index element={<DashboardHome />} />
  <Route path="stats" element={<Stats />} />
  <Route path="settings" element={<Settings />} />
</Route>

// DashboardLayout.jsx
function DashboardLayout() {
  return (
    <div className="dashboard">
      <aside>…sidebar links…</aside>
      <main><Outlet /></main>   {/* child route renders here */}
    </div>
  );
}
```

14.6 Protected Routes

```
function RequireAuth({ children }) {
  const { user } = useAuth();                    // from your Context
  if (!user) return <Navigate to="/login" replace />;
  return children;
}

<Route path="/dashboard" element={
  <RequireAuth><Dashboard /></RequireAuth>
} />
```

Exercises 14.

1. Build a three-page site (Home / About / Contact) with a navbar and a 404 page.
2. Add a blog: a list page linking to `/posts/:id` detail pages that fetch the post.
3. Build a dashboard with a persistent sidebar using nested routes and `<Outlet />`.
4. Add a fake login (Context) and protect the dashboard; redirect logged-in users away from `/login`.

KEY TAKEAWAYS

- React Router maps URLs to components without page reloads.
- `<Link>` / `<NavLink>` for navigation, `useParams` for dynamic segments, `useNavigate` for code-driven moves.
- `<Outlet />` enables shared layouts with nested routes.
- Protected routes are just wrapper components that redirect.

CHAPTER 15

State Management at Scale

15.1 Classifying Your State

Before reaching for a library, classify what you are storing — most "global state" is not global at all:

Table 15-1 Kinds of state and the right tool for each

Kind	Examples	Right tool
Local UI state	open modals, form fields, hover	`useState`
Shared client state	theme, current user, cart	Context + `useReducer`, Zustand, Redux
Server state	API data, cache, pagination	TanStack Query / SWR
URL state	filters, current page, tabs	React Router search params

15.2 Zustand — Minimal Global State

Zustand is a tiny, hook-based store — a great first step beyond Context:

```
npm install zustand
```

```
import { create } from "zustand";

const useCartStore = create((set) => ({
  items: [],
  addItem: (product) => set((s) => ({ items: [...s.items, product] })),
  clear: () => set({ items: [] }),
}));

// Any component, no Provider needed:
function CartBadge() {
  const count = useCartStore((s) => s.items.length);  // selector = precise re-
renders
  return <span>Cart: {count}</span>;
}
```

15.3 Redux Toolkit — Structured Global State

For large teams and complex domains, **Redux Toolkit (RTK)** is the industry standard: one immutable store, updates via dispatched actions, superb devtools (time-travel debugging).

```
npm install @reduxjs/toolkit react-redux
```

```
// counterSlice.js — a "slice" = reducer + actions together
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    incremented: (state) => { state.value += 1; },        // Immer makes this safe
    addedAmount: (state, action) => { state.value += action.payload; },
  },
});
export const { incremented, addedAmount } = counterSlice.actions;
export default counterSlice.reducer;

// store.js
import { configureStore } from "@reduxjs/toolkit";
export const store = configureStore({ reducer: { counter: counterSlice.reducer } });

// main.jsx
<Provider store={store}><App /></Provider>

// Any component:
import { useSelector, useDispatch } from "react-redux";

function Counter() {
  const value = useSelector((s) => s.counter.value);
  const dispatch = useDispatch();
  return <button onClick={() => dispatch(incremented())}>{value}</button>;
}
```

> **Definition.** Redux Toolkit's `createSlice` uses the *Immer* library internally, so you may write "mutating" code (`state.value += 1`) — Immer converts it into safe immutable updates.

15.4 Choosing an Approach

- Start with `useState` + lifting state. Most of an app never needs more.
- Add Context + `useReducer` for a few widely-shared values.

- Adopt **Zustand** when client state spreads across many distant components.
- Adopt **Redux Toolkit** for large teams, complex update logic, or when devtools/middleware matter.
- Always keep *server data* out of these stores — that is TanStack Query's job.

Exercises 15.

1. Rebuild your Chapter 12 cart with Zustand — compare the boilerplate.
2. Rebuild it again with Redux Toolkit; open Redux DevTools and time-travel through dispatches.
3. Write a one-page memo: for a given app idea, which state lives where (local / global / server / URL)? Defend your choices.

KEY TAKEAWAYS

- Classify state first: local, shared client, server, URL.
- Zustand: minimal stores, no provider, selector-based re-renders.
- Redux Toolkit: slices, immutable updates via Immer, world-class devtools.
- Server state belongs in a query library, not in Redux/Zustand.

CHAPTER 16

Performance Optimization

16.1 First: Don't Optimize Blindly

React is fast by default. Optimize only when you have a measured problem, and measure with the right tools: the **React DevTools Profiler** (records renders and their durations) and the browser's Performance tab.

16.2 Understand Why Components Re-render

1. Their own state changes.
2. A prop they receive changes (by reference comparison).
3. Their *parent re-renders* — children re-render by default, even with identical props.
4. A context they consume changes value.

Most performance work is about stopping unnecessary cases of #2 and #3.

16.3 React.memo — Skip Unnecessary Child Renders

```
import { memo } from "react";

const ExpensiveRow = memo(function ExpensiveRow({ item, onSelect }) {
  console.log("rendered", item.id);
  return <li onClick={() => onSelect(item.id)}>{item.name}</li>;
});
```

`memo` re-renders the child only when its props actually change. But props are compared by reference — a fresh inline function defeats it. This is where `useCallback` (Chapter 11) pairs in:

```
const handleSelect = useCallback((id) => setSelected(id), []);
// stable reference → memoized rows truly skip re-rendering
```

16.4 List Virtualization

Rendering 10,000 rows is slow no matter how well you memoize. **Virtualization** renders only the visible rows:

```
npm install @tanstack/react-virtual
// or: npm install react-window
```

The library renders ~30 DOM nodes for a 100,000-item list and recycles them as you scroll. This is the single biggest win for long lists.

16.5 Code Splitting & Lazy Loading

Ship less JavaScript up front: load heavy screens only when visited.

```
import { lazy, Suspense } from "react";

const Dashboard = lazy(() => import("./Dashboard"));

<Route path="/dashboard" element={
  <Suspense fallback={<p>Loading page…</p>}>
    <Dashboard />
  </Suspense>
} />
```

Vite automatically splits the bundle at each `import()` boundary.

16.6 Quick Wins Checklist

- Keep state as *low* in the tree as possible — a parent's state change re-renders everything below.
- Split context by concern; a single mega-context re-renders the world on every change.
- Avoid creating new object/array literals in JSX props for memoized children — hoist or memoize them.
- Use production builds (`npm run build`) — dev React is intentionally slower.

Exercises 16.

1. Build a parent with a counter and a memoized child list; add console logs and verify the list skips re-renders when the counter changes.
2. Break it on purpose: pass an inline arrow function to the memoized child, observe re-renders return, then fix with `useCallback` .
3. Render 10,000 rows naively, note the jank, then fix with virtualization.
4. Lazy-load a heavy "Reports" route and inspect the network tab to see the split chunk.

KEY TAKEAWAYS

- Measure first with React DevTools Profiler; don't guess.
- memo + useCallback + useMemo stop unnecessary re-renders.
- Virtualize long lists; lazy-load heavy routes.
- State placement and context design are architectural performance tools.

CHAPTER 17

Testing React Applications

17.1 Why Test?

Tests are executable documentation and a safety net for refactoring. In React, the modern philosophy is: **test behavior the way users experience it** — click buttons, read text — not implementation details like state variable names.

17.2 The Toolchain

- **Vitest** — the test runner (Jest-compatible API, Vite-native).
- **React Testing Library (RTL)** — renders components and queries the DOM like a user.
- **user-event** — simulates realistic typing and clicking.

```
npm install -D vitest @testing-library/react @testing-library/user-event @testing-
library/jest-dom jsdom
```

17.3 Your First Component Test

```
// Counter.test.jsx
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import Counter from "./Counter";

test("increments when the button is clicked", async () => {
  const user = userEvent.setup();
  render(<Counter />);

  const button = screen.getByRole("button", { name: /increment/i });
  await user.click(button);
  await user.click(button);

  expect(screen.getByText(/count: 2/i)).toBeInTheDocument();
});
```

The rhythm of every test: **Arrange** (render) → **Act** (user events) → **Assert** (what's on screen).

17.4 Querying Like a User

RTL offers several query families. Prefer them in this order:

Table 17-1 Testing Library query priority

Query	Finds by	Example
`getByRole`	Accessible role (button, heading, textbox)	`getByRole("button", { name: "Save" })`
`getByLabelText`	Form label	`getByLabelText("Email")`
`getByPlaceholderText`	Placeholder	`getByPlaceholderText("Search…")`
`getByText`	Visible text	`getByText(/welcome/i)`
`getByTestId`	`data-testid` attribute	Last resort only

Query prefixes: `getBy` throws if absent; `queryBy` returns null (use to assert absence: `expect(screen.queryByText("Error")).toBeNull()`); `findBy` is async and waits (use for data that appears after a fetch).

17.5 Testing Async Behavior & Mocking

```
test("shows users after loading", async () => {
  // Mock the network — tests must never hit real APIs
  vi.stubGlobal("fetch", vi.fn(() =>
    Promise.resolve({
      ok: true,
      json: () => Promise.resolve([{ id: 1, name: "Ada" }]),
    })
  ));

  render(<UserList />);
  expect(screen.getByText(/loading/i)).toBeInTheDocument();

  // findBy* waits for the async update
  expect(await screen.findByText("Ada")).toBeInTheDocument();
  vi.unstubAllGlobals();
});
```

Exercises 17.

1. Test your counter: initial render, increment, reset.
2. Test a login form: invalid email shows an error; valid input enables submit.
3. Test a to-do list: adding, toggling, and deleting items.
4. Test `UserList` with a mocked fetch, including the error path (mock `ok: false`).

KEY TAKEAWAYS

- Test user-visible behavior, not implementation details.
- Vitest + Testing Library + user-event is the standard stack.
- Arrange → Act → Assert; query by role and accessible names.
- Mock the network; use `findBy*` for async UI.

PART VII

Capstone Project

Combine every skill from the course into one portfolio-ready application.

CHAPTER 18

Capstone Project — TaskFlow Dashboard

18.1 The Brief

You will build **TaskFlow**, a personal task-management dashboard. It must feel like a real product: multiple pages, persistent data, async behavior, and polished UX. Work in milestones — each maps to course chapters.

Table 18-1 Capstone milestones mapped to chapters

Milestone	Deliverable	Chapters used
M1 — Shell	Vite project, routing, layout with sidebar	2, 14
M2 — Task CRUD	Add / complete / delete tasks, global store	4, 6, 7, 12
M3 — Forms	Task form with validation, filters via URL	8, 14
M4 — Persistence	localStorage sync via custom hook; mock API for "team" page	9, 10, 13
M5 — Polish	Styling, memoization of the list, lazy-loaded stats page	5, 16
M6 — Quality	Component tests, reducer tests, README	17

18.2 Suggested Architecture

```
src/
├── main.jsx                # Router + providers
├── App.jsx                 # Route definitions
├── components/
│   ├── Layout.jsx          # Sidebar + <Outlet />
│   ├── TaskItem.jsx        # memoized row
│   ├── TaskForm.jsx        # controlled, validated form
│   └── FilterBar.jsx       # writes filters to URL params
├── hooks/
│   ├── useLocalStorage.js
│   └── useFetch.js
├── store/
│   └── taskStore.js        # Context + useReducer (or Zustand)
├── pages/
│   ├── TasksPage.jsx
│   ├── StatsPage.jsx       # lazy-loaded
│   └── TeamPage.jsx        # fetches from mock API
└── tests/
    ├── TaskForm.test.jsx
    └── taskReducer.test.js
```

18.3 The Store (Starting Point)

```
// store/taskStore.js
import { createContext, useContext, useReducer } from "react";
import useLocalStorage from "../hooks/useLocalStorage";

function taskReducer(state, action) {
  switch (action.type) {
    case "added":
      return [...state, { id: crypto.randomUUID(), title: action.title, done: false
}];
    case "toggled":
      return state.map((t) => (t.id === action.id ? { ...t, done: !t.done } : t));
    case "deleted":
      return state.filter((t) => t.id !== action.id);
    default:
      throw new Error("Unknown action " + action.type);
  }
}

const TaskContext = createContext(null);

export function TaskProvider({ children }) {
  const [persisted, setPersisted] = useLocalStorage("taskflow.tasks", []);
  const [tasks, dispatch] = useReducer(taskReducer, persisted);

  // keep localStorage in sync with reducer state
  const value = {
    tasks,
    dispatch: (action) => {
      const next = taskReducer(tasks, action);
      setPersisted(next);
      dispatch(action);
    },
  };
  return <TaskContext.Provider value={value}>{children}</TaskContext.Provider>;
}

export function useTasks() {
  return useContext(TaskContext);
}
```

18.4 Feature Checklist (Rubric)

Table 18-2 Grading rubric — 100 points

Feature	Points

Routing: /tasks, /stats, /team, 404; shared layout	15
Task CRUD with global store; empty states	20
Validated task form; filters synced to URL	15
localStorage persistence via custom hook	10
Team page: fetch, loading, error, retry	15
Performance: memoized TaskItem, lazy /stats	10
Tests: reducer + one component test; clean README	15

18.5 Stretch Goals

- Drag-and-drop reordering (with stable keys, of course).
- Dark mode via Context.
- Replace the mock API with a real backend (or Firebase/Supabase).
- Deploy to Vercel/Netlify and add the live URL to the README.

KEY TAKEAWAYS

- Real apps are compositions of small patterns you already know.
- Build in milestones; get something working at each step.
- The rubric is your contract — check it before you submit.

APPENDIX A

JavaScript You Need for React

Most "React problems" beginners hit are actually JavaScript gaps. Master these eight features — they appear in literally every React file.

A.1 Arrow Functions

```
const add = (a, b) => a + b;
const double = (n) => n * 2;
button.addEventListener("click", () => console.log("hi"));
```

A.2 Destructuring

```
const user = { name: "Ada", age: 36 };
const { name, age } = user;                    // object destructuring (props!)
const [first, ...rest] = [1, 2, 3];            // array destructuring (useState!)
```

A.3 Spread & Rest

```
const copy = { ...user, age: 37 };             // immutable object update
const list = [...oldList, newItem];            // immutable array add
```

A.4 Template Literals

```
const cls = `btn ${active ? "btn-active" : ""}`;
```

A.5 Array Methods — the React Big Four

Table A-1 Array methods used constantly in React

Method	Returns	React use
`map(fn)`	new array, transformed	rendering lists
`filter(fn)`	new array, subset	removing items, filtered views
`find(fn)`	first match or undefined	lookup by id

`reduce(fn, init)`	single accumulated value	totals, grouping

A.6 Ternary & Short-Circuit

```
const label = isAdmin ? "Admin" : "User";
isLoggedIn && <Dashboard />;                 // render-if pattern
const name = user?.profile?.name ?? "Guest";  // optional chaining + nullish
coalescing
```

A.7 Modules

```
export default function App() {}              // one default export per file
export const helper = () => {};               // named exports
import App from "./App";
import { helper } from "./utils";
```

A.8 Promises & async/await

```
async function load() {
  try {
    const res = await fetch("/api/data");
    const data = await res.json();
    return data;
  } catch (err) {
    console.error(err);
  }
}
```

APPENDIX B

Hooks Cheat Sheet

Table B-1 Every hook from this course on one page

Hook	Signature	Purpose	Chapter
`useState`	`[value, setValue] = useState(init)`	Local component state	6
`useEffect`	`useEffect(setup, deps)`	Sync with external systems	9
`useRef`	`ref = useRef(init)`	DOM access; mutable render-invisible value	11
`useMemo`	`cached = useMemo(fn, deps)`	Cache expensive calculations	11
`useCallback`	`fn = useCallback(fn, deps)`	Cache function identity	11
`useReducer`	`[state, dispatch] = useReducer(reducer, init)`	Centralized state transitions	12
`useContext`	`value = useContext(MyContext)`	Read broadcast values	12
custom hooks	`useWhatever(...)`	Reusable stateful logic	13

The Rules of Hooks: (1) call only at the top level — no conditions, loops, or nested functions; (2) call only from React function components or other hooks.

APPENDIX C

Resources & Next Steps

C.1 Official Resources

- **react.dev** — the official documentation; its "Learn" section and interactive sandboxes are superb for students.
- **vite.dev** — Vite documentation.
- **reactrouter.com** — React Router docs.
- **redux-toolkit.js.org**, **zustand.docs.pmnd.rs**, **tanstack.com/query** — ecosystem libraries used in this course.

C.2 Where to Go After This Course

1. **TypeScript** — the industry default for React codebases; typed props catch bugs at compile time.
2. **Next.js** — the dominant full-stack React framework: server rendering, file-based routing, API routes.
3. **TanStack Query** — professional server-state management.
4. **Tailwind CSS** — utility-first styling used across the industry.
5. **React Native / Expo** — take your skills to iOS and Android.

C.3 How to Keep Growing

- Build projects without tutorials — struggle is where learning happens.
- Read other people's code on GitHub.
- Rebuild classic UIs (a calculator, Trello board, chat app) from scratch.
- Teach what you learn — you are holding proof that it works.

Congratulations on completing the course. From an empty folder to a tested, optimized, routed, globally-stateful application — that is the zero-to-hero journey, and you walked it. Now go build something people love.